

AI Evals Reference Guide — Tài liệu hệ thống hóa nguyên tắc đánh giá AI

Tài liệu tham khảo Khoá Vĩn AI Thực Chiến - Track 1 - Day 21+22

1. Tư duy nền tảng

1.1. Đánh giá AI không chỉ là đo usage

Với phần mềm truyền thống, team thường đo conversion, retention, funnel completion, click-through, hoặc time-on-task. Những chỉ số này cho biết người dùng có đi qua flow hay không.

Với sản phẩm AI, câu hỏi chính khác đi:

- AI có hiểu đúng intent của người dùng không?
- AI có làm đúng phần việc được giao không?
- AI có ra quyết định hợp lý trong điều kiện thiếu thông tin không?
- AI có tuân thủ policy, constraint, permission và business rule không?
- Khi AI sai, lỗi đó có chấp nhận được không?

Vì vậy, eval không chỉ đo “người dùng có dùng tính năng không”, mà đo **chất lượng phần việc AI thực hiện**.

1.2. Đánh giá model khác với đánh giá application

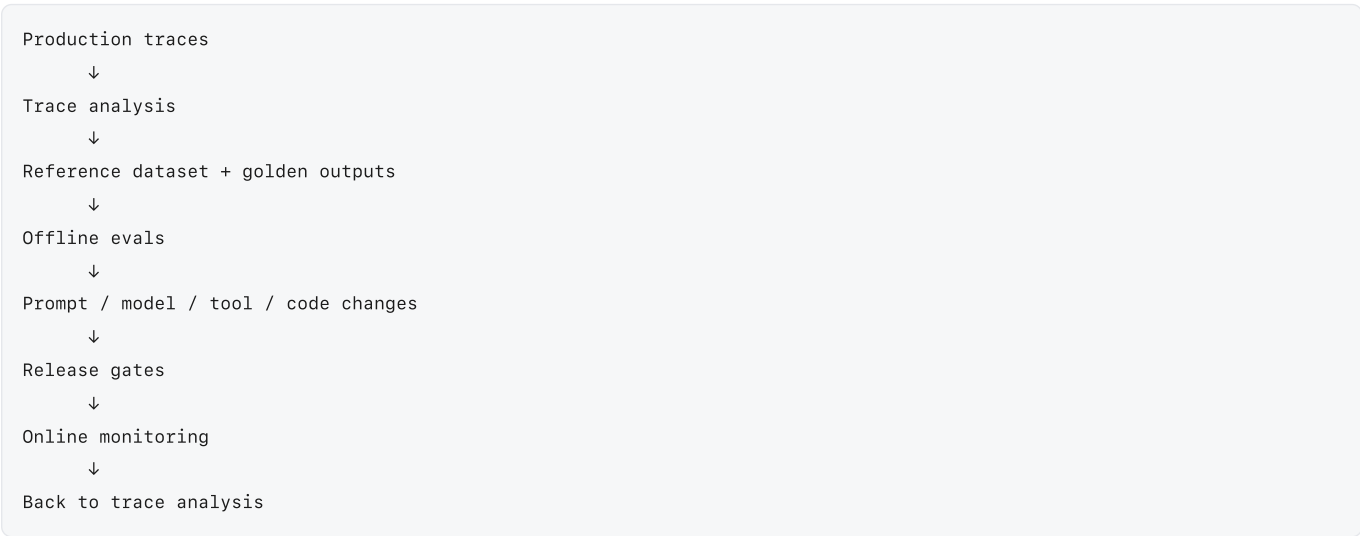
Có hai lớp đánh giá khác nhau:

Lớp	Câu hỏi chính	AI thường sở hữu	Ví dụ
Model eval	Model có reasoning, instruction-following, safety, coding, math tốt không?	Model provider, research team	MMLU, coding benchmark, safety benchmark
Application eval	Trong sản phẩm này, với người dùng này, trong context này, output có đủ tốt không?	Product + engineering + domain expert	Support triage accuracy, RAG answer usefulness, code suggestion accepted rate

Tài liệu này tập trung vào **application eval**, vì đây là nơi benchmark chung không thể định nghĩa thay bạn thế nào là “đúng”, “tốt”, “đủ an toàn” hay “có thể ship”.

1.3. AI Flywheel

Một hệ thống AI tốt cần một vòng lặp cải thiện liên tục:



Năm thành phần cốt lõi:

1. **Agent Success Rate** — north star metric đo chất lượng công việc AI.
2. **Trace Analysis** — đọc log, session, conversation, tool calls để tìm failure pattern.
3. **Reference Dataset** — tập case có golden output, edge case và expected behavior.

- 4. **Offline Evals** — test tự động trước release.
- 5. **Online Monitoring** — giám sát production, drift, regression, user complaints.

2. Ba tầng eval: codebase, con người, LLM

Một hệ thống eval tốt không chọn duy nhất một cách chấm. Nó phối hợp ba nguồn đánh giá:

- 1. **Codebase / deterministic checks**
- 2. **Human evaluation**
- 3. **LLM-as-judge / model-based evaluation**

Cách hiểu ngắn gọn:

Nguồn đánh giá	Dùng khi	Ưu điểm	Rủi ro
Codebase	Đúng/sai rõ ràng, kiểm tra được bằng rule, DB, schema, API, regex, numeric threshold	Nhanh, rẻ, ổn định, chạy trong CI	Không chấm được chất lượng ngữ nghĩa phức tạp
Con người	Cần judgment, domain expertise, policy nuance, high-stakes decision, rubric chưa ổn định	Chuẩn nhất cho “good” trong domain	Chậm, đắt, khó scale, có bias giữa người chấm
LLM	Cần chấm ngữ nghĩa ở scale, sau khi đã có rubric và calibration với con người	Scale tốt, rẻ hơn human, hữu ích cho triage/critique	Có thể lệch chuẩn, cần đo precision/recall và kiểm tra định kỳ

3. Nguyên tắc ra quyết định: dùng codebase, con người hay LLM?

3.1. Cây quyết định nhanh

Câu hỏi cần chấm có thể được xác minh bằng rule/code không?

└ Có → dùng codebase assertion trước.

└ Ví dụ: JSON hợp lệ, enum đúng, không lộ UUID, giá trị nằm trong DB, permission đúng.

└ Không → câu hỏi có cần domain judgment, empathy, policy nuance hoặc high-stakes không?

└ Có → dùng con người làm nguồn chuẩn, ít nhất cho labeling/calibration.

└ Ví dụ: phản hồi có làm khách hàng giận hơn không, medical/legal nuance, severity của lỗi.

└ Không hoặc đã có rubric ổn định → dùng LLM-as-judge có calibration.

└ Ví dụ: phân loại intent, đánh giá answer có grounded không, critique style/clarity.

3.2. Rule of thumb

- **Codebase trước** nếu có thể biến tiêu chí thành assertion.
- **Con người trước** khi chưa biết “good” nghĩa là gì.
- **LLM sau** khi đã có rubric, examples và human-labeled calibration set.
- **Không dùng LLM để thay thế hoàn toàn con người** trong giai đoạn đầu hoặc trong domain high-stakes.
- **Không dùng human để chấm thứ code có thể chấm được**, vì sẽ chậm, đắt và dễ inconsistency.
- **Không dùng code để giả vờ chấm judgment**, ví dụ ép `overall_quality_score >= 0.8` khi score này do heuristic yếu tạo ra.

4. Khi nào dùng codebase?

Dùng codebase khi tiêu chí có thể kiểm tra bằng logic xác định, dữ liệu nguồn đáng tin cậy hoặc invariant kỹ thuật.

4.1. Các nhóm tiêu chí phù hợp với codebase

Nhóm	Ví dụ check	Gợi ý implementation
Schema	Output có đúng JSON schema không?	JSON schema validator, Zod, Pydantic
Enum	<code>category</code> phải thuộc <code>Technical</code> , <code>Billing</code> , <code>Feature Request</code>	assertion

Nhóm	Ví dụ check	Gợi ý implementation
Format	Không lộ UUID, email, token, internal ID	regex
Permission	User có quyền xem record này không?	policy engine, RBAC check
Tool usage	Agent có gọi đúng tool không? Có gọi tool cấm không?	trace parser
Data correctness	ID, price, status, count có khớp DB không?	DB query
Retrieval	Retrieved docs có chứa source bắt buộc không?	metadata check
Latency/cost	P95 latency < threshold, token cost < budget	metrics pipeline
Safety hard rule	Không trả lời ngoài phạm vi, không gửi email khi chưa confirm	state machine / guardrail
Regression	Case từng pass không được fail sau prompt/model change	CI eval runner

4.2. Ví dụ: schema cho Support Triage Agent

```
{
  "ticket_id": "TCK-123",
  "category": "Billing",
  "sentiment": "Frustrated",
  "needs_human": true,
  "urgency": "High",
  "reason": "User asks to cancel after being charged twice."
}
```

Schema mong muốn:

```
{
  "type": "object",
  "required": ["ticket_id", "category", "sentiment", "needs_human", "urgency", "reason"],
  "properties": {
    "ticket_id": { "type": "string" },
    "category": {
      "type": "string",
      "enum": ["Technical", "Billing", "Feature Request", "Clarification Needed"]
    },
    "sentiment": {
      "type": "string",
      "enum": ["Positive", "Neutral", "Frustrated", "Angry"]
    },
    "needs_human": { "type": "boolean" },
    "urgency": {
      "type": ["string", "null"],
      "enum": ["High", "Critical", null]
    },
    "reason": { "type": "string" }
  }
}
```

4.3. Ví dụ code: TypeScript assertion

```
type Category = "Technical" | "Billing" | "Feature Request" | "Clarification Needed";
type Sentiment = "Positive" | "Neutral" | "Frustrated" | "Angry";
type Urgency = "High" | "Critical" | null;

interface TriageOutput {
  ticket_id: string;
  category: Category;
  sentiment: Sentiment;
  needs_human: boolean;
  urgency: Urgency;
  reason: string;
}

const CATEGORIES = new Set(["Technical", "Billing", "Feature Request", "Clarification Needed"]);
const SENTIMENTS = new Set(["Positive", "Neutral", "Frustrated", "Angry"]);
const URGENCIES = new Set(["High", "Critical", null]);

export function assertValidTriageOutput(output: TriageOutput) {
  if (!output.ticket_id) throw new Error("missing_ticket_id");
  if (!CATEGORIES.has(output.category)) throw new Error(`invalid_category:${output.category}`);
  if (!SENTIMENTS.has(output.sentiment)) throw new Error(`invalid_sentiment:${output.sentiment}`);
  if (!URGENCIES.has(output.urgency)) throw new Error(`invalid_urgency:${output.urgency}`);
  if (!output.reason || output.reason.length < 10) throw new Error("weak_reason");

  if (["Frustrated", "Angry"].includes(output.sentiment) && output.needs_human !== true) {
    throw new Error("frustrated_or_angry_must_escalate");
  }

  if (output.needs_human && output.urgency === null) {
    throw new Error("escalated_ticket_must_have_urgency");
  }
}
```

4.4. Ví dụ code: không lộ internal ID

```
export function assertNoInternalIds(text: string) {
  const withoutTemplateBlocks = text.replace(/\{\{.*?\}\}/g, "");
  const uuidRegex = /[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}/gi;
  const matches = withoutTemplateBlocks.match(uuidRegex) ?? [];

  if (matches.length > 0) {
    throw new Error(`exposed_uuid:${matches.join(",")}`);
  }
}
```

4.5. Ví dụ code: check RAG grounding bằng source metadata

```
def assert_required_sources_present(answer, retrieved_docs, required_doc_ids):
    retrieved_ids = {doc["doc_id"] for doc in retrieved_docs}
    missing = set(required_doc_ids) - retrieved_ids
    if missing:
        raise AssertionError(f"missing_required_sources:{sorted(missing)}")

    if not answer.get("citations"):
        raise AssertionError("answer_missing_citations")

    cited_ids = {c["doc_id"] for c in answer["citations"]}
    invalid_citations = cited_ids - retrieved_ids
    if invalid_citations:
        raise AssertionError(f"citation_not_in_context:{sorted(invalid_citations)}")
```

4.6. Khi không nên dùng codebase làm judge chính

Không nên dùng codebase nếu tiêu chí là:

- “Câu trả lời có hữu ích không?”
- “Giọng văn có đủ đồng cảm không?”
- “Agent có hiểu đúng ý ngầm không?”
- “Output có đủ tốt để khách hàng enterprise chấp nhận không?”
- “Lỗi này nghiêm trọng tới mức nào?”

Những tiêu chí này cần human rubric hoặc LLM judge đã được calibration.

5. Khi nào dùng con người?

Dùng con người khi tiêu chí đánh giá cần judgment mà code hoặc LLM chưa thể đáng tin.

5.1. Các tình huống nên dùng human eval

Tình huống	Vì sao cần con người
Giai đoạn prototype	Chưa biết failure modes, chưa có rubric ổn định
Định nghĩa golden outputs	Cần chuẩn chất lượng do team/domain expert xác nhận
High-stakes domain	Lỗi có thể gây thiệt hại lớn: pháp lý, y tế, tài chính, bảo mật
Policy nuance	Cần hiểu ngoại lệ, ngữ cảnh, ranh giới chấp nhận
Subjective quality	Empathy, helpfulness, tone, product taste
Label calibration	Cần chuẩn để đo LLM judge có lệch không
Debug lỗi mới	Cần phân tích nguyên nhân gốc và severity
Drift	Production xuất hiện pattern mới chưa có trong dataset

5.2. Human eval không nhất thiết phải phức tạp

Bắt đầu đơn giản:

```
✓ Good / có thể ship
~ Needs minor fix / cần chỉnh nhẹ
× Bad / không chấp nhận được
```

Sau đó thêm lý do:

```
{
  "human_label": "bad",
  "failure_mode": "wrong_category",
  "severity": "P1",
  "human_notes": "Ticket nói bị double charge nhưng agent phân loại Feature Request.",
  "golden_category": "Billing",
  "golden_sentiment": "Frustrated",
  "golden_needs_human": true
}
```

5.3. Vai trò của con người trong từng giai đoạn

Giai đoạn	Human làm gì?	Output
Prototype / vibe check	Đọc 10–30 case đa dạng, ghi pass/fail, phát hiện failure pattern	Seed dataset, initial rubric
Build / offline eval	Label golden outputs, review fail cases, approve thresholds	Reference dataset, release gate
Optimize / monitoring	Sample production traces, audit LLM judge, tìm drift	New edge cases, updated rubric

5.4. Human eval fields nên lưu

```
{
  "trace_id": "trace_abc123",
  "eval_case_id": "support_triage_042",
  "reviewer_id": "domain_expert_01",
  "reviewed_at": "2026-06-26T10:00:00+07:00",
  "human_outcome": "fail",
  "severity": "P1",
  "failure_modes": ["wrong_category", "missed_escalation"],
  "expected_behavior": "Classify as Billing and escalate to human with High urgency.",
  "reviewer_notes": "Customer is clearly frustrated about being charged twice.",
  "is_golden": true
}
```

5.5. Cách lấy mẫu human eval

Không nên chỉ random hoàn toàn. Nên phối hợp nhiều loại sample:

Sample type	Mục đích
Random production sample	Ước lượng chất lượng tổng thể
High-risk sample	Audit các case có tiền, privacy, safety, angry user
Low-confidence sample	Chấm các case LLM judge không chắc
Disagreement sample	Human vs LLM judge lệch nhau
New intent sample	Phát hiện nhu cầu mới
Regression sample	Kiểm tra lỗi cũ có quay lại không

6. Khi nào dùng LLM-as-judge?

Dùng LLM khi bạn cần chấm nhiều output ở scale, tiêu chí có tính ngữ nghĩa, và đã có rubric đủ rõ.

6.1. LLM phù hợp để chấm gì?

Nhiệm vụ	Phù hợp?	Ghi chú
Intent classification	Có	Cần enum rõ và examples
Sentiment / frustration detection	Có	Cần test sarcasm, mixed intent

Nhiệm vụ	Phù hợp?	Ghi chú
Groundedness trong RAG	Có, nhưng phải kết hợp code	Code check citations, LLM check semantic support
Helpfulness	Có, nếu rubric rõ	Cần human calibration
Tone / empathy	Có	Cần examples tốt/xấu
Policy compliance mềm	Có, nếu policy được đưa vào judge prompt	High-risk vẫn cần human audit
Exact factual correctness với DB	Không nên dùng một mình	Dùng code/DB làm source of truth
Security / permission	Không nên dùng một mình	Dùng code policy check
Release blocking high-stakes	Không nên dùng một mình	Cần human hoặc deterministic gate

6.2. Điều kiện trước khi dùng LLM judge

Trước khi dùng LLM judge ở quy mô lớn, cần có:

- 1. Rubric rõ.
- 2. Examples pass/fail.
- 3. Human-labeled calibration set.
- 4. Đo precision/recall theo từng failure mode.
- 5. Cơ chế audit định kỳ.
- 6. Versioning cho judge prompt/model.

6.3. Không chỉ đo raw agreement

Nếu 95% case là pass, một judge luôn đoán pass cũng có 95% agreement. Vì vậy, cần đo:

- Precision: trong các case judge nói fail, bao nhiêu case thật sự fail?
- Recall: trong các case thật sự fail, judge bắt được bao nhiêu?
- False positive rate: judge gán fail quá nhiều không?
- False negative rate: judge bỏ sót lỗi nghiêm trọng không?

Ví dụ:

```
def precision_recall(human_labels, judge_labels, positive_label="fail"):
    tp = sum(h == positive_label and j == positive_label for h, j in zip(human_labels, judge_labels))
    fp = sum(h != positive_label and j == positive_label for h, j in zip(human_labels, judge_labels))
    fn = sum(h == positive_label and j != positive_label for h, j in zip(human_labels, judge_labels))

    precision = tp / (tp + fp) if tp + fp else 0
    recall = tp / (tp + fn) if tp + fn else 0
    return {"precision": precision, "recall": recall}
```

6.4. Ví dụ LLM judge prompt

You are evaluating the output of a Support Ticket Triage Agent.

Task:

Decide whether the agent output is acceptable for production.

Rubric:

- PASS if category, sentiment, escalation decision, and urgency are all reasonable.
- FAIL if the category is wrong in a way that routes the ticket to the wrong team.
- FAIL if a frustrated or angry user is not escalated.
- FAIL if the agent invents information not present in the ticket.
- PASS_WITH_NOTES if the output is usable but reason text could be clearer.

Allowed labels:

- pass
- pass_with_notes
- fail

Ticket:

{{ticket_text}}

Agent output:

{{agent_output}}

Return JSON only:

```
{
  "label": "pass | pass_with_notes | fail",
  "failure_modes": ["wrong_category", "wrong_sentiment", "missed_escalation", "hallucination", "bad_reasoning"],
  "critique": "short explanation",
  "confidence": 0.0
}
```

6.5. Ví dụ output của LLM judge

```
{
  "label": "fail",
  "failure_modes": ["wrong_category", "missed_escalation"],
  "critique": "The user reports a duplicate charge and is clearly frustrated, but the agent classified the ticket as Feature Request.",
  "confidence": 0.92
}
```

6.6. Khi LLM judge phải chuyển cho human

```
def route_judge_result(judge_result, case):
    high_risk = case["risk_level"] in ["high", "critical"]
    low_confidence = judge_result["confidence"] < 0.75
    severe_failure = any(
        mode in judge_result["failure_modes"]
        for mode in ["privacy_leak", "unsafe_advice", "missed_escalation"]
    )

    if high_risk or low_confidence or severe_failure:
        return "human_review"

    return "accept_llm_judge"
```

7. Eval lifecycle: từ prototype đến production

7.1. Giai đoạn 1 — Vibe Check

Mục tiêu: hiểu AI làm được gì, fail ở đâu, và “good” nên được định nghĩa thế nào.

Workflow:

1. Tạo 10–30 input đa dạng.
2. Chạy qua prototype.
3. Human review output.
4. Gắn nhãn ✓ / ~ / ✕.
5. Ghi failure mode.
6. Chọn candidate golden outputs.
7. Cập nhật PRD/rubric.

Ví dụ field:

```
{
  "case_id": "vibe_support_001",
  "input": "Help! I was charged twice and nobody is answering me.",
  "prototype_output": {
    "category": "Technical",
    "sentiment": "Neutral",
    "needs_human": false
  },
  "human_label": "fail",
  "failure_modes": ["wrong_category", "wrong_sentiment", "missed_escalation"],
  "expected_output": {
    "category": "Billing",
    "sentiment": "Frustrated",
    "needs_human": true,
    "urgency": "High"
  },
  "notes": "Short input but enough evidence for billing + escalation."
}
```

7.2. Giai đoạn 2 — Offline Evals

Mục tiêu: trước mỗi release, biết version mới tốt hơn, tệ hơn hay chỉ khác đi.

Workflow:

```
Prompt/model/tool/code change
↓
Run eval suite on reference dataset
↓
Run deterministic assertions
↓
Run LLM judge where needed
↓
Compare to baseline
↓
Apply release gate
↓
Ship or block
```

Release gate mẫu:

```
release_gate:
  block_if:
    - p0_failures > 0
    - p1_failures > baseline_p1_failures
    - schema_pass_rate < 0.995
    - category_accuracy < 0.90
    - escalation_recall < 0.95
    - human_review_required_cases > 20
  warn_if:
    - average_latency_ms > baseline_latency_ms * 1.2
    - average_cost_usd > baseline_cost_usd * 1.15
    - llm_judge_disagreement_rate > 0.10
```

7.3. Giai đoạn 3 — Online Monitoring

Mục tiêu: bắt drift, lỗi mới, thay đổi hành vi người dùng và regression production.

Monitoring fields:

```
{
  "trace_id": "trace_prod_789",
  "user_id_hash": "u_anon_456",
  "feature": "support_triage",
  "variant": "prompt_v12_model_b",
  "input_intent": "billing_refund",
  "agent_success_signal": {
    "user_feedback": "thumbs_down",
    "user_action": "reopened_ticket",
    "semantic_signal": "repeated_request"
  },
  "llm_monitor_label": "fail",
  "human_audit_label": null,
  "latency_ms": 1840,
  "cost_usd": 0.0031,
  "created_at": "2026-06-26T10:00:00+07:00"
}
```

Online monitoring nên đưa case mới quay lại reference dataset nếu:

- Có failure mode mới.
- Có intent mới.
- LLM judge và human disagree.
- User complaint tăng nhưng offline metric vẫn tốt.
- Case đại diện cho segment quan trọng.

8. Thiết kế reference dataset

8.1. Dataset phải đại diện cho work, không chỉ prompt

Một eval case tốt mô tả đầy đủ:

- User input.
- Context hệ thống.
- Tool/database state.
- Expected output hoặc rubric.
- Failure modes cần bắt.
- Severity nếu fail.
- Segment / persona / intent.

Schema mẫu:

```
{
  "eval_case_id": "support_triage_050",
  "feature": "support_triage",
  "intent": "billing_duplicate_charge",
  "persona": "angry_paid_customer",
  "input": "I was charged twice this month. Fix this now or I cancel.",
  "context": {
    "plan": "Pro",
    "account_status": "active",
    "recent_invoices": ["inv_001", "inv_002"]
  },
  "expected": {
    "category": "Billing",
    "sentiment": "Angry",
    "needs_human": true,
    "urgency": "Critical"
  },
  "assertions": [
    "valid_schema",
    "category_matches_expected",
    "angry_user_must_escalate"
  ],
  "judge_rubric_id": "support_triage_rubric_v3",
  "severity_if_failed": "P1",
  "source": "production_trace",
  "is_golden": true
}
```

8.2. Các loại case cần có

Case type	Ví dụ	Lý do cần có
Happy path	Ticket billing rõ ràng	Đảm bảo core behavior
Ambiguous	“Help!”	Bắt hallucination, cần clarification
Multi-intent	“I want refund and discount”	Bắt việc rơi mất intent
Edge policy	User đòi cập nhật payment info	Bắt privacy/security
Angry/sarcasm	“Great, your app broke again”	Bắt sentiment nuance
Tool failure	CRM timeout	Bắt fallback behavior
Empty/low info	“Hello”	Bắt refusal/clarification
Regression	Lỗi từng xảy ra	Ngăn lỗi quay lại

8.3. Dataset size theo giai đoạn

Giai đoạn	Size gợi ý	Mục tiêu
Vibe check	10–30	Tạo trực giác, tìm failure pattern
Initial offline eval	50–100	Release gate đầu tiên
Mature offline eval	200–1000+	Phù segment, regression, edge cases
Monitoring sample	1–10% traffic hoặc risk-based sample	Bắt drift và lỗi mới

Con số không phải luật cứng. Dataset tốt là dataset giúp team ra quyết định chính xác hơn, không phải dataset to nhất.

9. Thiết kế metric

9.1. Agent Success Rate

Agent Success Rate nên là composite metric, kết hợp nhiều tín hiệu:

```
Agent Success Rate = f(
    task_success,
    user_feedback,
    user_action,
    semantic_quality,
    policy_compliance,
    human_or_llm_judge_score
)
```

Ví dụ cho support triage:

```
def compute_agent_success(row):
    if row["p0_failure"]:
        return 0.0

    score = 0.0

    if row["category_correct"]:
        score += 0.30
    if row["sentiment_correct"]:
        score += 0.20
    if row["escalation_correct"]:
        score += 0.30
    if row["reason_quality"] == "good":
        score += 0.10
    if row["user_feedback"] == "positive":
        score += 0.10

    return min(score, 1.0)
```

9.2. Không dùng một metric duy nhất cho mọi thứ

Nên theo dõi nhiều metric theo lớp:

Metric	Nguồn	Dùng để
Schema pass rate	Code	Bắt lỗi format
Task accuracy	Code/human/LLM	Đo core task
Escalation recall	Code/human	Bắt lỗi bỏ sót case cần người
Groundedness	Code + LLM	Đo RAG
Helpfulness	Human + LLM	Đo UX quality
P0/P1 count	Human/code	Release gate
Latency/cost	Observability	Kiểm soát vận hành
Drift rate	Online monitoring	Phát hiện production shift

9.3. Metric phải segment được

Không chỉ xem overall pass rate. Cần xem theo:

- Intent.
- Persona.
- Language.
- Customer tier.

- Tool path.
- Prompt version.
- Model version.
- Source: synthetic / historical / production.
- Risk level.

Ví dụ SQL:

```
select
  model_version,
  prompt_version,
  intent,
  count(*) as n_cases,
  avg(case when final_outcome = 'pass' then 1 else 0 end) as pass_rate,
  sum(case when severity = 'P0' then 1 else 0 end) as p0_count,
  sum(case when severity = 'P1' then 1 else 0 end) as p1_count
from eval_results
where eval_suite = 'support_triage_release_gate'
group by model_version, prompt_version, intent
order by p1_count desc, pass_rate asc;
```

10. Eval architecture đề xuất

10.1. Các bảng dữ liệu chính

eval_cases

```
create table eval_cases (
  eval_case_id text primary key,
  feature text not null,
  intent text,
  persona text,
  risk_level text,
  input jsonb not null,
  context jsonb,
  expected jsonb,
  rubric_id text,
  source text,
  is_golden boolean default false,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);
```

eval_runs

```
create table eval_runs (
  eval_run_id text primary key,
  eval_suite text not null,
  model_version text,
  prompt_version text,
  code_version text,
  judge_model_version text,
  judge_prompt_version text,
  started_at timestamptz default now(),
  completed_at timestamptz,
  status text
);
```

eval_results

```
create table eval_results (  
  eval_result_id text primary key,  
  eval_run_id text references eval_runs(eval_run_id),  
  eval_case_id text references eval_cases(eval_case_id),  
  output jsonb,  
  deterministic_checks jsonb,  
  llm_judge_result jsonb,  
  human_review_result jsonb,  
  final_outcome text,  
  severity text,  
  failure_modes text[],  
  latency_ms integer,  
  cost_usd numeric,  
  created_at timestamptz default now()  
);
```

10.2. Eval runner pseudo-code

```
def run_eval_suite(eval_cases, candidate_system, judge=None):
    results = []

    for case in eval_cases:
        output = candidate_system.run(
            input=case["input"],
            context=case.get("context")
        )

        deterministic = run_deterministic_checks(case, output)

        if deterministic["has_blocking_failure"]:
            final = {
                "outcome": "fail",
                "source": "codebase",
                "severity": deterministic["max_severity"],
                "failure_modes": deterministic["failure_modes"]
            }
        elif case.get("requires_human_review"):
            final = {
                "outcome": "pending_human_review",
                "source": "human"
            }
        elif judge:
            judge_result = judge.evaluate(case, output)
            final = route_llm_judge(case, judge_result)
        else:
            final = {
                "outcome": "pass",
                "source": "codebase"
            }

        results.append({
            "case_id": case["eval_case_id"],
            "output": output,
            "deterministic": deterministic,
            "final": final
        })

    return summarize_results(results)
```

10.3. Final outcome precedence

Khi nhiều nguồn đánh giá cùng tồn tại, nên có thứ tự ưu tiên:

```
P0 deterministic failure
> P0 human failure
> P0 LLM judge failure with high confidence
> human label
> deterministic pass/fail
> calibrated LLM judge
> unknown
```

Ví dụ logic:

```
def resolve_final_outcome(code_result, human_result, llm_result):
    if code_result.has_p0:
        return "fail_p0"

    if human_result and human_result.outcome == "fail":
        return f"fail_{human_result.severity.lower()}"

    if llm_result and llm_result.label == "fail" and llm_result.confidence >= 0.9:
        if "privacy_leak" in llm_result.failure_modes:
            return "fail_p0"
        return "fail"

    if code_result.has_failure:
        return "fail"

    if llm_result and llm_result.label == "pass":
        return "pass"

    return "needs_review"
```

11. Ví dụ end-to-end: Support Ticket Triage

11.1. Product task

Agent tự động phân loại ticket support:

- `category` : Technical / Billing / Feature Request / Clarification Needed
- `sentiment` : Positive / Neutral / Frustrated / Angry
- `needs_human` : true / false
- `urgency` : High / Critical / null

11.2. Eval contract

```
feature: support_triage
version: v1
must:
  - output_valid_json
  - category_in_allowed_enum
  - sentiment_in_allowed_enum
  - angry_or_frustrated_user_must_escalate
  - no_internal_ids_or_private_notes
should:
  - reason_mentions_evidence_from_ticket
  - ambiguous_ticket_should_use_clarification_needed
  - multi_intent_ticket_should_preserve_primary_and_secondary_intents
must_not:
  - invent_account_history
  - mark_angry_user_as_neutral
  - route_billing_issue_to_feature_request
```

11.3. Test case

```
{
  "eval_case_id": "support_triage_101",
  "input": {
    "ticket_id": "TCK-101",
    "body": "I hate this app. I was charged twice and I want to cancel unless someone fixes it today."
  },
  "expected": {
    "category": "Billing",
    "sentiment": "Angry",
    "needs_human": true,
    "urgency": "Critical"
  },
  "severity_if_failed": "P1",
  "risk_level": "high"
}
```

11.4. Deterministic checks

```
def check_support_triage(case, output):
    errors = []

    allowed_categories = {"Technical", "Billing", "Feature Request", "Clarification Needed"}
    allowed_sentiments = {"Positive", "Neutral", "Frustrated", "Angry"}

    if output["category"] not in allowed_categories:
        errors.append("invalid_category")

    if output["sentiment"] not in allowed_sentiments:
        errors.append("invalid_sentiment")

    if output["sentiment"] in {"Frustrated", "Angry"} and not output["needs_human"]:
        errors.append("missed_escalation")

    if output["needs_human"] and output.get("urgency") not in {"High", "Critical"}:
        errors.append("missing_urgency")

    expected = case.get("expected", {})
    if expected.get("category") and output["category"] != expected["category"]:
        errors.append("category_mismatch")

    return errors
```

11.5. LLM judge dùng để chấm phần reason

Code có thể kiểm tra enum và escalation, nhưng khó chấm "reason có hợp lý không". Dùng LLM judge:

```
Evaluate whether the reason given by the triage agent is supported by the ticket.
Fail if the reason adds facts not present in the ticket.
Fail if the reason ignores the main complaint.
Pass if the reason is concise and cites the key evidence.
```

11.6. Human review dùng khi nào?

Chuyển human nếu:

- Ticket thuộc enterprise customer.
- Sentiment là Angry nhưng AI confidence thấp.
- LLM judge và deterministic checks mâu thuẫn.
- Output bị user thumbs down.

- Case đại diện failure mode mới.

12. Ví dụ end-to-end: RAG Q&A

12.1. Product task

AI trả lời câu hỏi dựa trên knowledge base.

12.2. Eval dimensions

Dimension	Nguồn chấm chính
Retrieved docs đúng không?	Code + human sample
Answer có citation không?	Code
Citation có nằm trong context không?	Code
Answer có được support bởi citation không?	LLM + human calibration
Answer có hữu ích không?	LLM + human
Answer có nói “không biết” khi thiếu bằng chứng không?	LLM + code rule

12.3. Eval case

```
{
  "eval_case_id": "rag_policy_022",
  "question": "Can I get a refund after 45 days?",
  "required_doc_ids": ["refund_policy_v4"],
  "expected_behavior": "Answer that refunds are normally available within 30 days, and advise contacting support for excep
  "must_not": ["claim_refund_is_always_available", "omit_time_window", "no_citation"]
}
```

12.4. Judge prompt

```
You are evaluating a RAG answer.

Question:
{{question}}

Retrieved context:
{{context}}

Answer:
{{answer}}

Rubric:
- Fail if the answer states a fact not supported by the retrieved context.
- Fail if the answer omits a policy constraint that changes the meaning.
- Fail if the answer gives a definitive answer when the context is insufficient.
- Pass if the answer is grounded, cites the relevant source, and is helpful.

Return JSON:
{
  "label": "pass | fail",
  "groundedness": "grounded | partially_grounding | ungrounded",
  "missing_constraints": [],
  "unsupported_claims": [],
  "critique": "...",
  "confidence": 0.0
}
```

13. Ví dụ end-to-end: Coding Assistant

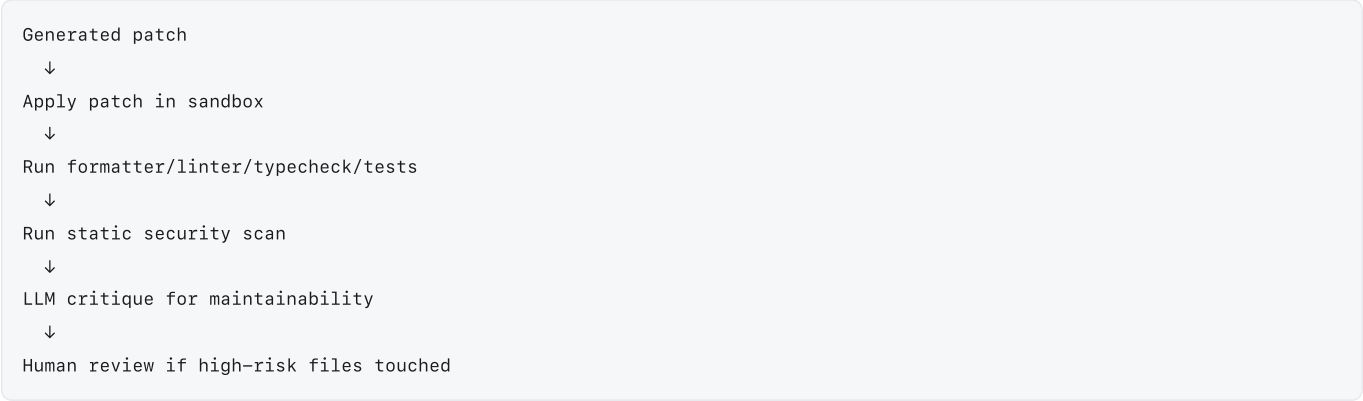
13.1. Product task

AI đề xuất code hoặc sửa bug.

13.2. Eval dimensions

Dimension	Nguồn chấm chính
Code compile không?	Codebase
Unit tests pass không?	Codebase
Typecheck/lint pass không?	Codebase
Không phá public API?	Codebase + human review
Giải pháp có maintainable không?	Human + LLM
Có security issue không?	Code scanner + human
Có giải đúng intent không?	Human + LLM

13.3. Pipeline



13.4. Example gate

```
coding_assistant_gate:
  block_if:
    - tests_failed > 0
    - typecheck_failed == true
    - security_scan_severity in ["critical", "high"]
    - touched_files contains ["auth/", "billing/", "permissions/"] and human_review_missing == true
  warn_if:
    - llm_maintainability_score < 0.7
    - patch_size_lines > 300
```

14. Failure mode taxonomy

Nên chuẩn hóa tên lỗi để dễ phân tích.

```
failure_modes:
  task_understanding:
    - wrong_intent
    - missed_intent
    - wrong_category
    - ambiguity_not_handled
  factuality:
    - hallucination
    - unsupported_claim
    - stale_information
    - wrong_calculation
  tool_use:
    - wrong_tool
    - missing_tool_call
    - unnecessary_tool_call
    - tool_result_ignored
  policy:
    - privacy_leak
    - permission_violation
    - unsafe_advice
    - missed_escalation
  output_quality:
    - invalid_schema
    - bad_formatting
    - unclear_reasoning
    - poor_tone
    - too_verbos
    - not_actionable
  reliability:
    - timeout
    - high_latency
    - high_cost
    - nondeterministic_regression
```

Severity:

```
severity:
P0: user harm, privacy/security leak, illegal/unsafe action, money movement without authorization
P1: wrong routing, missed escalation, materially wrong answer, enterprise-visible severe issue
P2: confusing but recoverable, minor factual miss, formatting issue affecting usability
P3: style nit, verbosity, minor tone issue
```

15. Decision matrix tổng hợp

Câu hỏi đánh giá	Codebase	Human	LLM	Ghi chú
Output đúng schema không?	Primary	No	No	Chạy mọi request nếu có thể
Output có enum hợp lệ không?	Primary	No	No	Deterministic
Có lộ UUID/token/internal ID không?	Primary	Audit	No	Regex + scanner
Có dùng đúng permission không?	Primary	Audit	No	Không giao cho LLM
Có gọi đúng tool không?	Primary	Debug	Maybe	LLM chỉ hỗ trợ critique
Category đúng không?	Code nếu có label	Human để tạo label	LLM sau calibration	Tùy domain

Câu hỏi đánh giá	Codebase	Human	LLM	Ghi chú
Sentiment đúng không?	No	Calibration	Primary sau calibration	Cần edge cases
Reason có hợp lý không?	No	Calibration	Primary	Subjective semantic
Answer có grounded không?	Partial	Calibration	Primary	Code check citation, LLM check meaning
Tone có phù hợp không?	No	Primary ban đầu	Scale sau	Cần examples
Có nên escalate human không?	Partial	Primary cho high-risk	Secondary	Recall quan trọng hơn precision
Có thể release không?	Gate metrics	Approve exception	Advisory	Release decision không nên chỉ do LLM
Có drift production không?	Metrics	Audit	Monitor	Kết hợp cả ba

16. Checklist triển khai từ zero

Tuần 1: Vibe check

- ☐ Chọn một AI task cụ thể.
- ☐ Viết prototype prompt đơn giản.
- ☐ Tạo 10–30 inputs đa dạng.
- ☐ Chạy prototype.
- ☐ Human review với ✓ / ~ / ✕.
- ☐ Ghi failure modes.
- ☐ Viết rubric v0.
- ☐ Chọn golden outputs đầu tiên.

Tuần 2: Reference dataset + code assertions

- ☐ Tạo `eval_cases` JSON/CSV.
- ☐ Thêm expected labels hoặc expected behavior.
- ☐ Viết schema validator.
- ☐ Viết enum/regex/security checks.
- ☐ Viết eval runner đơn giản.
- ☐ Lưu kết quả theo model/prompt/code version.

Tuần 3: Offline eval gate

- ☐ Chạy eval trong CI hoặc pre-release job.
- ☐ So sánh candidate với baseline.
- ☐ Đặt threshold block/warn.
- ☐ Tạo dashboard pass rate, P0/P1, latency, cost.
- ☐ Review fail cases sau mỗi run.

Tuần 4: LLM judge + human calibration

- ☐ Viết judge rubric.
- ☐ Tạo calibration set có human labels.
- ☐ Đo precision/recall của judge.
- ☐ Điều chỉnh judge prompt.
- ☐ Chỉ dùng judge cho dimensions đạt chuẩn.
- ☐ Thiết lập human audit định kỳ.

Sau launch: Online monitoring

- ☐ Log traces đầy đủ.
- ☐ Sample production sessions.
- ☐ Chấm risk-based sample.
- ☐ Theo dõi user feedback + semantic signals.

- ☐ Thêm failure modes mới vào taxonomy.
 - ☐ Cập nhật reference dataset.
 - ☐ Theo dõi drift giữa offline và online.
-

17. Anti-patterns cần tránh

17.1. Chỉ vibe check rồi ship

Vibe check giúp hiểu vấn đề, nhưng không đủ để regression test. Sau khi có prototype, phải chuyển thành dataset và offline eval.

17.2. Chỉ có một eval "overall correctness"

Một score chung không cho biết lỗi nằm ở đâu. Cần tách theo dimension: intent, grounding, tool use, policy, tone, format, latency, cost.

17.3. Dùng LLM judge mà không calibration

LLM judge cũng là một model cần được đánh giá. Phải so với human labels và đo precision/recall.

17.4. Để human chấm quá nhiều thứ deterministic

Nếu lỗi có thể bắt bằng code, hãy bắt bằng code. Human nên dành cho judgment.

17.5. Không đọc trace

Không có trace analysis thì team chỉ đo những gì mình đã biết. Production sẽ sinh ra intent và edge case mới.

17.6. Dataset không được cập nhật

Reference dataset phải sống. Mỗi lỗi production quan trọng nên trở thành regression case.

17.7. Offline metric tăng nhưng complaint cũng tăng

Đây là dấu hiệu eval đang đo sai thứ. Cần đọc traces, segment metrics và cập nhật rubric/dataset.

18. Template PRD cho AI eval

```
# AI Eval Contract: <Feature Name>

## 1. AI work unit
AI chịu trách nhiệm thực hiện phần việc gì?

## 2. User/context
Ai dùng? Context nào? Constraint nào quan trọng?

## 3. Output contract
Fields bắt buộc, schema, allowed enum, format.

## 4. Quality dimensions
- Correctness
- Groundedness
- Helpfulness
- Safety/policy
- Tone
- Tool usage
- Latency/cost

## 5. Must / should / must not
### Must
- ...

### Should
- ...

### Must not
- ...

## 6. Failure modes
Tên lỗi, định nghĩa, severity.

## 7. Reference dataset
Nguồn case, size, coverage, owner.

## 8. Eval methods
| Dimension | Codebase | Human | LLM |
| --- | --- | --- | --- |
| ... | ... | ... | ... |

## 9. Release gate
Block/warn thresholds.

## 10. Monitoring plan
Sampling, dashboard, audit cadence, drift policy.
```

19. Kết luận thực dụng

Nếu phải rút gọn thành một nguyên tắc:

Codebase chấm cái chắc chắn. Con người định nghĩa cái đúng. LLM scale cái đã được con người định nghĩa và kiểm định.

Một hệ thống eval tốt không phải là một dashboard đẹp hay một framework phức tạp. Nó là khả năng trả lời nhanh và đáng tin các câu hỏi:

- Version mới có tốt hơn version cũ không?

- Nó tốt hơn ở segment nào, tệ hơn ở segment nào?
- Lỗi nghiêm trọng nhất là gì?
- Có thể ship không?
- Nếu không ship, cần sửa prompt, model, tool, retrieval, policy hay code?
- Production có đang drift khỏi offline dataset không?

Khi eval làm tốt, team sẽ iterate nhanh hơn, debug nhanh hơn, release tự tin hơn và biến feedback thực tế thành improvement liên tục.